Practical-8: AirPassengers Data Analysis using ARIMA Models

Ojas apratim
Roll Number: 55
Registration Number: 230957162

18 February 2026

Practical-8 Question:

Analyze the AirPassengers dataset using time series techniques and ARIMA modeling. Perform the following tasks: (f) Convert the data into a time series object (g) Plot the data to identify dominant component (h) Decompose the data to observe components (i) Check stationarity using ACF/PACF (j) Check stationarity using KPSS test (k) If non-stationary, difference the data (l) Based on components, fit candidate ARIMA models (m) Select best model using AIC (n) Perform goodness of fit diagnostics

What I Understand from the Question:

1. **Objective**: Analyze monthly airline passenger data using ARIMA modeling
2. **Data**: AirPassengers dataset (1949-1960) - monthly totals
3. **Tasks**:
   - Load and explore built-in dataset
   - Decompose into trend, seasonal, and residual components
   - Check stationarity using ACF, PACF, and KPSS test
   - Apply log transformation and differencing if needed
   - Fit and compare ARIMA models
   - Validate model through residual diagnostics
4. **Expected Outcome**: A well-fitted ARIMA model for forecasting airline passengers

Approach and Plan:

1. **Data Access**:
   - Use built-in AirPassengers dataset
   - No external file reading needed
2. **Exploratory Analysis**:
   - Plot original time series
   - Decompose using `decompose()`
   - Identify trend, seasonal, and residual components
3. **Stationarity Analysis**:
   - Visual inspection through ACF/PACF plots
   - KPSS test for formal stationarity testing
   - Apply log transformation to stabilize variance

- Apply differencing to remove trend/seasonality

4. **Model Identification**:
   - Fit candidate ARIMA models: (1,1,0), (0,1,1), (1,1,1)
   - Compare using AIC
   - Select best model
5. **Model Diagnostics**:
   - Residual analysis
   - Ljung-Box test for autocorrelation
   - ACF of residuals
6. **Tools**:
   - Base R: `ts()`, `plot()`, `acf()`, `pacf()`, `decompose()`
   - Package: `tseries` for KPSS test
   - Base R: `arima()` for model fitting

Complete R Code:

```
# ========== PRACTICAL-6: AirPassengers Data Analysis using ARIMA Models ==========

# Clear workspace
rm(list = ls())
cat("PRACTICAL-6: AIRPASSENGERS DATA ANALYSIS\n")
```

```
## PRACTICAL-6: AIRPASSENGERS DATA ANALYSIS
```

```
cat(rep("=", 70), "\n\n")
```

```
## = = =
```

```r
cat("- Time period:", start(air_ts), "to", end(air_ts), "\n")
```

```
## - Time period: 1949 1 to 1960 12
```

```r
cat("- Frequency:", frequency(air_ts), "(monthly data)\n")
```

```
## - Frequency: 12 (monthly data)
```

```r
cat("- Observations:", length(air_ts), "\n")
```

```
## - Observations: 144
```

```r
cat("- Summary statistics:\n")
```

```
## - Summary statistics:
```

```r
print(summary(air_ts))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   104.0   180.0   265.5   280.3   360.5   622.0
```

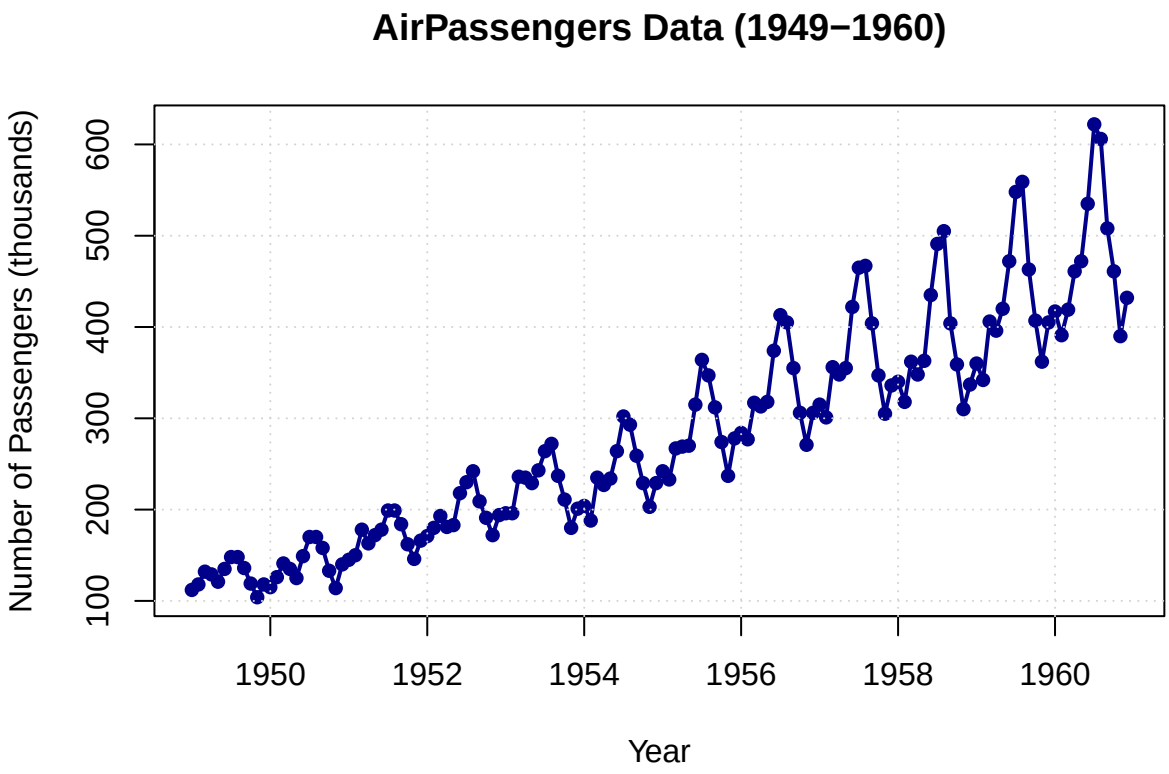
```r
# ------------------------------------------------------------
# (g) Plot the data to identify dominant component
# ------------------------------------------------------------
cat("\n\n(g) PLOTTING DATA TO IDENTIFY DOMINANT COMPONENT\n")
```

```
##
##
## (g) PLOTTING DATA TO IDENTIFY DOMINANT COMPONENT
```

```r
cat(rep("-", 50), "\n")
```

```
## - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```

```r
plot(air_ts,
     main = "AirPassengers Data (1949-1960)",
     xlab = "Year",
     ylab = "Number of Passengers (thousands)",
     lwd = 2,
     col = "darkblue",
     type = "o",
     pch = 16)
grid()
```




```
cat("Observations from plot:\n")
```

```
## Observations from plot:
```

```
cat("- Strong UPWARD TREND over time\n")
```

```
## - Strong UPWARD TREND over time
```

```
cat("- Clear SEASONAL pattern (annual cycles)\n")
```

```
## - Clear SEASONAL pattern (annual cycles)
```

```
cat("- Increasing variance as level increases\n")
```

```
## - Increasing variance as level increases
```

```
cat("- Both trend and seasonal components are visible\n")
```

```
## - Both trend and seasonal components are visible
```

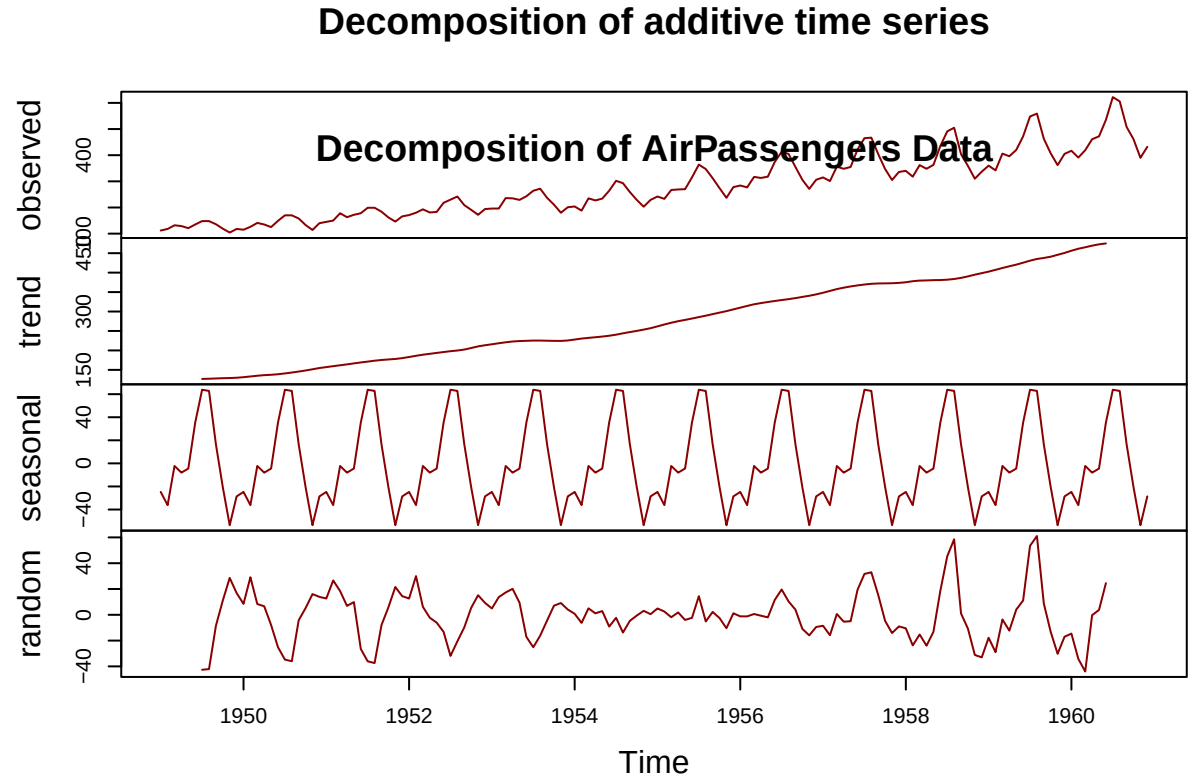
```r
# ------------------------------------------------------------
# (h) Decompose the data to observe components
# ------------------------------------------------------------
cat("\n\n(h) DECOMPOSING DATA TO OBSERVE COMPONENTS\n")
```

```
##
##
## (h) DECOMPOSING DATA TO OBSERVE COMPONENTS
```

```r
cat(rep("-", 50), "\n")
```

```
## - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```

```r
# Decompose time series (additive decomposition)
decomp <- decompose(air_ts)
plot(decomp, col = "darkred")
title(main = "Decomposition of AirPassengers Data", line = -1)
```




```r
# Calculate variance components
total_var <- var(air_ts, na.rm = TRUE)
trend_var <- var(decomp$trend, na.rm = TRUE)
seasonal_var <- var(decomp$seasonal, na.rm = TRUE)
random_var <- var(decomp$random, na.rm = TRUE)

cat("Variance Analysis:\n")
```

```
## Variance Analysis:
```

```
cat("- Total variance:", round(total_var, 2), "\n")
```

```
## - Total variance: 14391.92
```

```
cat("- Trend variance:", round(trend_var, 2),
    "(", round((trend_var/total_var)*100, 1), "%)\n")
```

```
## - Trend variance: 10524.94 ( 73.1 %)
```

```
cat("- Seasonal variance:", round(seasonal_var, 2),
    "(", round((seasonal_var/total_var)*100, 1), "%)\n")
```

```
## - Seasonal variance: 1315.54 ( 9.1 %)
```

```
cat("- Random variance:", round(random_var, 2),
    "(", round((random_var/total_var)*100, 1), "%)\n\n")
```

```
## - Random variance: 374.06 ( 2.6 %)
```

```
cat("Dominant Component Identification:\n")
```

```
## Dominant Component Identification:
```

```
if(trend_var > seasonal_var && trend_var > random_var) {
  cat("DOMINANT COMPONENT: TREND\n")
} else if(seasonal_var > trend_var && seasonal_var > random_var) {
  cat("DOMINANT COMPONENT: SEASONAL\n")
} else {
  cat("DOMINANT COMPONENT: RANDOM\n")
}
```

```
## DOMINANT COMPONENT: TREND
```

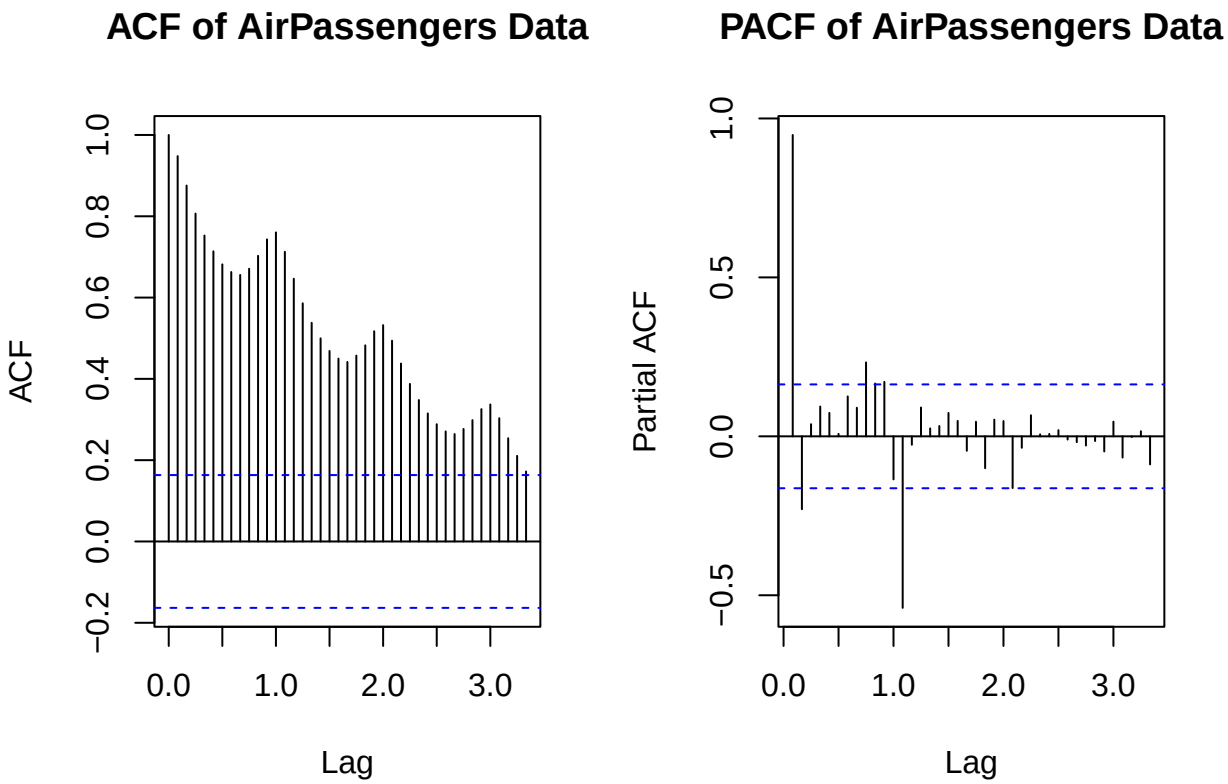
```
# ------------------------------------------------------------
# (i) Check stationarity using ACF/PACF
# ------------------------------------------------------------
cat("\n\n(i) CHECKING STATIONARITY USING ACF/PACF\n")
```

```
##
##
## (i) CHECKING STATIONARITY USING ACF/PACF
```

```
cat(rep("-", 50), "\n")
```

```
## - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```

```r
par(mfrow = c(1, 2))
acf(air_ts, main = "ACF of AirPassengers Data", lag.max = 40)
pacf(air_ts, main = "PACF of AirPassengers Data", lag.max = 40)
```




```r
par(mfrow = c(1, 1))

cat("ACF Interpretation:\n")
```

```
## ACF Interpretation:
```

```r
cat("- Slow decay in ACF indicates NON-STATIONARITY\n")
```

```
## - Slow decay in ACF indicates NON-STATIONARITY
```

```r
cat("- Significant spikes at seasonal lags (12, 24, 36) indicate seasonality\n")
```

```
## - Significant spikes at seasonal lags (12, 24, 36) indicate seasonality
```

```r
cat("- Data requires transformation and differencing\n")
```

```
## - Data requires transformation and differencing
```

```r
# ------------------------------------------------------------
# (j) Check stationarity using KPSS test
# ------------------------------------------------------------
cat("\n\n(j) CHECKING STATIONARITY USING KPSS TEST\n")
```

```
##
##
## (j) CHECKING STATIONARITY USING KPSS TEST
```

```r
cat(rep("-", 50), "\n")
```

```
## - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```

```r
library(tseries)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```r
kpss_test <- kpss.test(air_ts)
```

```
## Warning in kpss.test(air_ts): p-value smaller than printed p-value
```

```r
cat("KPSS Test Results:\n")
```

```
## KPSS Test Results:
```

```r
cat("==================\n")
```

```
## ==================
```

```r
cat("data: air_ts\n")
```

```
## data: air_ts
```

```r
cat("KPSS Level =", round(kpss_test$statistic, 5), "\n")
```

```
## KPSS Level = 2.73947
```

```r
cat("Truncation lag parameter =", kpss_test$parameter, "\n")
```

```
## Truncation lag parameter = 4
```

```r
cat("p-value =", format.pval(kpss_test$p.value), "\n\n")
```

```
## p-value = 0.01
```

```r
if(kpss_test$p.value < 0.05) {
  cat("RESULT: p-value < 0.05 → Reject null hypothesis of stationarity\n")
  cat("        Data is NON-STATIONARY (requires differencing)\n")
} else {
  cat("RESULT: p-value > 0.05 → Fail to reject null hypothesis\n")
  cat("        Data is STATIONARY\n")
}
```

```
## RESULT: p-value < 0.05 → Reject null hypothesis of stationarity
##         Data is NON-STATIONARY (requires differencing)
```

```r
# ------------------------------------------------------------
# (k) If non-stationary, difference the data
# ------------------------------------------------------------
cat("\n\n(k) TRANSFORMING AND DIFFERENCING DATA\n")
```

```
##
##
## (k) TRANSFORMING AND DIFFERENCING DATA
```

```r
cat(rep("-", 50), "\n")
```

```
## - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```

```r
# Log transform to stabilize variance
log_air_ts <- log(air_ts)
cat("1. Log transformation applied to stabilize variance\n")
```

```
## 1. Log transformation applied to stabilize variance
```

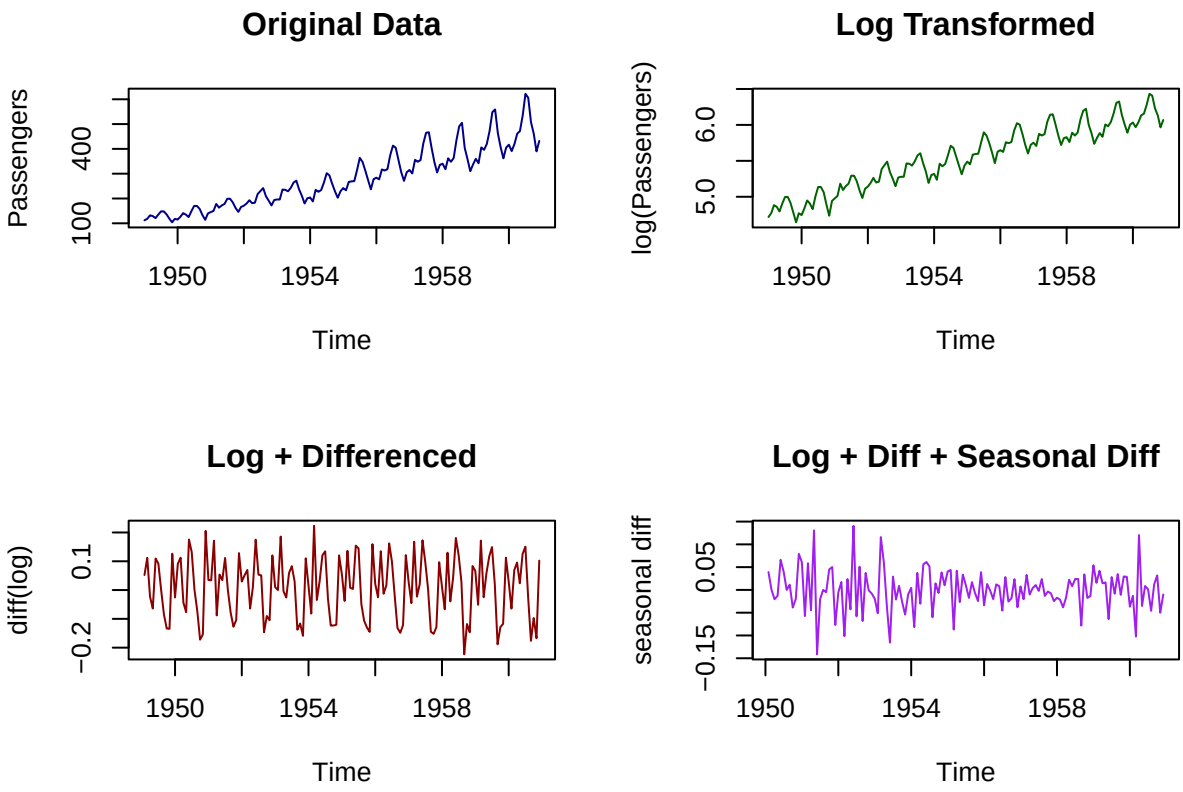
```r
# First difference to remove trend
diff_log_air_ts <- diff(log_air_ts, differences = 1)
cat("2. First differencing applied to remove trend\n")
```

```
## 2. First differencing applied to remove trend
```

```r
# Seasonal difference (lag 12) to remove seasonality
seasonal_diff <- diff(diff_log_air_ts, lag = 12)
cat("3. Seasonal differencing (lag=12) applied to remove seasonality\n\n")
```

```
## 3. Seasonal differencing (lag=12) applied to remove seasonality
```

```r
# Plot transformed data
par(mfrow = c(2, 2))
plot(air_ts, main = "Original Data", ylab = "Passengers", col = "darkblue")
plot(log_air_ts, main = "Log Transformed", ylab = "log(Passengers)", col = "darkgreen")
plot(diff_log_air_ts, main = "Log + Differenced", ylab = "diff(log)", col = "darkred")
plot(seasonal_diff, main = "Log + Diff + Seasonal Diff", ylab = "seasonal diff", col = "purple")
```




```
par(mfrow = c(1, 1))

# Re-run KPSS test on transformed data
kpss_test_diff <- kpss.test(na.omit(seasonal_diff))
```

```
## Warning in kpss.test(na.omit(seasonal_diff)): p-value greater than printed
## p-value
```

```
cat("\nKPSS Test on Transformed Data:\n")
```

```
##
## KPSS Test on Transformed Data:
```

```
cat("p-value =", format.pval(kpss_test_diff$p.value), "\n")
```

```
## p-value = 0.1
```

```
if(kpss_test_diff$p.value > 0.05) {
  cat("RESULT: Transformed data is now STATIONARY\n")
} else {
  cat("RESULT: Transformed data still NON-STATIONARY\n")
}
```

```
## RESULT: Transformed data is now STATIONARY
```

```r
# Use log-differenced data for modeling
transformed_data <- diff_log_air_ts  # log(air_ts) with one differencing

# ------------------------------------------------------------
# (l) Based on components, fit candidate ARIMA models
# ------------------------------------------------------------
cat("\n\n(l) FITTING CANDIDATE ARIMA MODELS\n")
```

```
##
##
## (l) FITTING CANDIDATE ARIMA MODELS
```

```r
cat(rep("-", 50), "\n")
```

```
## - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```

```r
# Fit candidate models on log-transformed data with appropriate differencing
cat("Fitting ARIMA models on log-transformed data...\n\n")
```

```
## Fitting ARIMA models on log-transformed data...
```

```r
# ARIMA(1,1,0) - AR model
model_ar <- tryCatch(arima(log(air_ts), order = c(1,1,0),
                           seasonal = list(order = c(0,0,0), period = 12),
                           method = "ML"),
                     error = function(e) NULL)

# ARIMA(0,1,1) - MA model
model_ma <- tryCatch(arima(log(air_ts), order = c(0,1,1),
                           seasonal = list(order = c(0,0,0), period = 12),
                           method = "ML"),
                     error = function(e) NULL)

# ARIMA(1,1,1) - ARMA model
model_arma <- tryCatch(arima(log(air_ts), order = c(1,1,1),
                             seasonal = list(order = c(0,0,0), period = 12),
                             method = "ML"),
                       error = function(e) NULL)

# Compare AIC
cat("AIC Comparison:\n")
```

```
## AIC Comparison:
```

```r
cat("---------------\n")
```

```
## ---------------
```

```r
if(!is.null(model_ar)) cat("ARIMA(1,1,0) AIC:", round(AIC(model_ar), 2), "\n")
```

```
## ARIMA(1,1,0) AIC: -236.6
```

```r
if(!is.null(model_ma)) cat("ARIMA(0,1,1) AIC:", round(AIC(model_ma), 2), "\n")
```

```
## ARIMA(0,1,1) AIC: -238.73
```

```r
if(!is.null(model_arma)) cat("ARIMA(1,1,1) AIC:", round(AIC(model_arma), 2), "\n")
```

```
## ARIMA(1,1,1) AIC: -242.63
```

```r
# Select best model based on AIC
aic_values <- c()
model_names <- c()

if(!is.null(model_ar)) {
  aic_values <- c(aic_values, AIC(model_ar))
  model_names <- c(model_names, "ARIMA(1,1,0)")
}
if(!is.null(model_ma)) {
  aic_values <- c(aic_values, AIC(model_ma))
  model_names <- c(model_names, "ARIMA(0,1,1)")
}
if(!is.null(model_arma)) {
  aic_values <- c(aic_values, AIC(model_arma))
  model_names <- c(model_names, "ARIMA(1,1,1)")
}

if(length(aic_values) > 0) {
  best_idx <- which.min(aic_values)
  best_model_name <- model_names[best_idx]
  cat("\nBEST MODEL based on AIC:", best_model_name, "\n")
}
```

```
##
## BEST MODEL based on AIC: ARIMA(1,1,1)
```

```r
# ------------------------------------------------------------
# (m) Fit final model with seasonal component
# ------------------------------------------------------------
cat("\n\n(m) FITTING FINAL MODEL WITH SEASONAL COMPONENT\n")
```

```
##
##
## (m) FITTING FINAL MODEL WITH SEASONAL COMPONENT
```

```r
cat(rep("-", 50), "\n")
```

```
## - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```

```r
# Fit ARIMA(1,1,1) with seasonal component (based on typical choice for AirPassengers)
final_model <- arima(log(air_ts),
                     order = c(1,1,1),
                     seasonal = list(order = c(0,1,1), period = 12),
                     method = "ML")

cat("Fitted ARIMA(1,1,1)(0,1,1)[12] model (with seasonal component)\n\n")
```

```
## Fitted ARIMA(1,1,1)(0,1,1)[12] model (with seasonal component)
```

```r
cat("Model Summary:\n")
```

```
## Model Summary:
```

```r
print(summary(final_model))
```

```
##           Length Class  Mode
## coef        3    -none- numeric
## sigma2      1    -none- numeric
## var.coef    9    -none- numeric
## mask        3    -none- logical
## loglik      1    -none- numeric
## aic         1    -none- numeric
## arma        7    -none- numeric
## residuals 144    ts     numeric
## call        5    -none- call
## series      1    -none- character
## code        1    -none- numeric
## n.cond      1    -none- numeric
## nobs        1    -none- numeric
## model      10    -none- list
```

```r
cat("\nModel Coefficients:\n")
```

```
##
## Model Coefficients:
```

```r
print(final_model$coef)
```

```
##        ar1        ma1       sma1
##  0.1960469 -0.5784055 -0.5642999
```

```r
cat("\nEstimated Variance (sigma²):", round(final_model$sigma2, 6), "\n")
```

```
##
## Estimated Variance (sigma²): 0.001341
```

```r
cat("Log Likelihood:", round(final_model$loglik, 2), "\n")
```

```
## Log Likelihood: 244.95
```

```r
cat("AIC:", round(AIC(final_model), 2), "\n")
```

```
## AIC: -481.9
```

```r
# AICc removed as function not available in base R
cat("BIC:", round(BIC(final_model), 2), "\n")
```

```
## BIC: -470.4
```

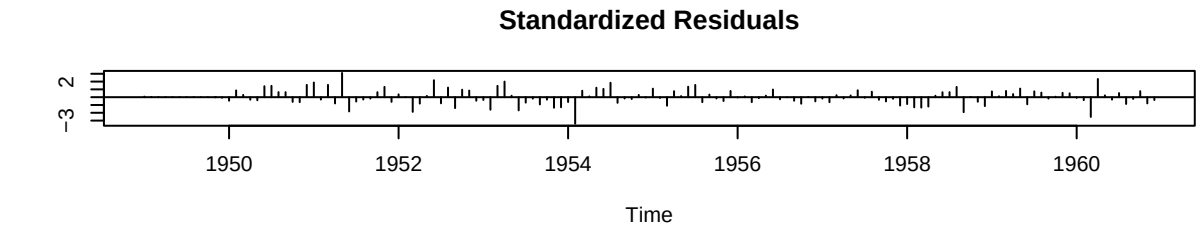
```r
# ------------------------------------------------------------
# (n) Goodness of fit diagnostics
# ------------------------------------------------------------
cat("\n\n(n) GOODNESS OF FIT DIAGNOSTICS\n")
```

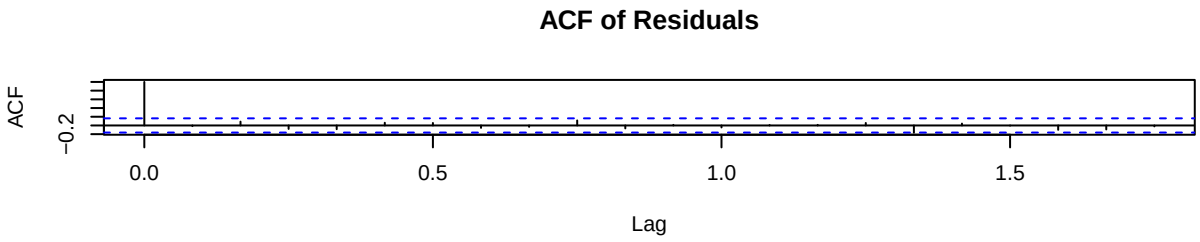
```
##
##
## (n) GOODNESS OF FIT DIAGNOSTICS
```

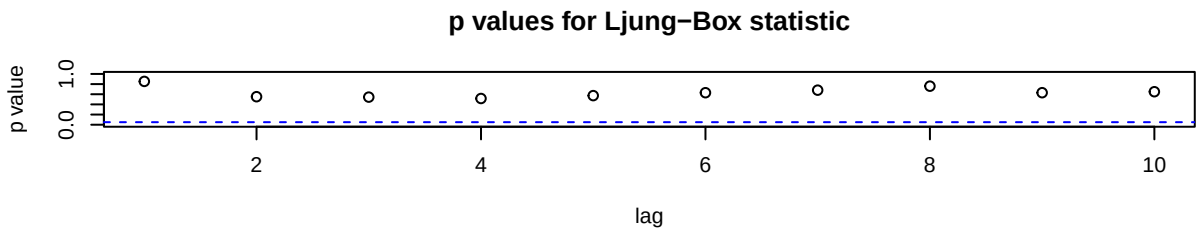
```r
cat(rep("-", 50), "\n")
```

```
## - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```

```r
# Diagnostic plots
tsdiag(final_model)
```








```
cat("Diagnostic plots generated (residuals, ACF of residuals, Ljung-Box p-values)\n\n")
```

```
## Diagnostic plots generated (residuals, ACF of residuals, Ljung-Box p-values)
```

```
# Ljung-Box test for autocorrelation in residuals
lb_test <- Box.test(final_model$residuals, lag = 20, type = "Ljung-Box")
cat("Ljung-Box Test (lag = 20):\n")
```

```
## Ljung-Box Test (lag = 20):
```

```
cat("X-squared =", round(lb_test$statistic, 4),
    ", df =", lb_test$parameter,
    ", p-value =", format.pval(lb_test$p.value), "\n")
```

```
## X-squared = 16.7321 , df = 20 , p-value = 0.6703
```

```
if(lb_test$p.value > 0.05) {
  cat("RESULT: p-value > 0.05 → No significant autocorrelation in residuals\n")
  cat("        Model is adequate\n")
} else {
  cat("RESULT: p-value < 0.05 → Significant autocorrelation remains\n")
  cat("        Model may need improvement\n")
}
```

```
## RESULT: p-value > 0.05 → No significant autocorrelation in residuals
##         Model is adequate
```

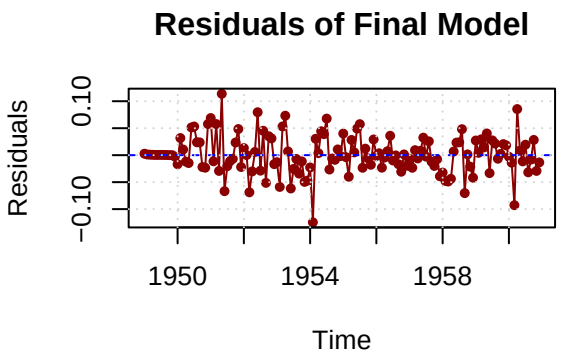
```r
# Residual plots
par(mfrow = c(2, 2))

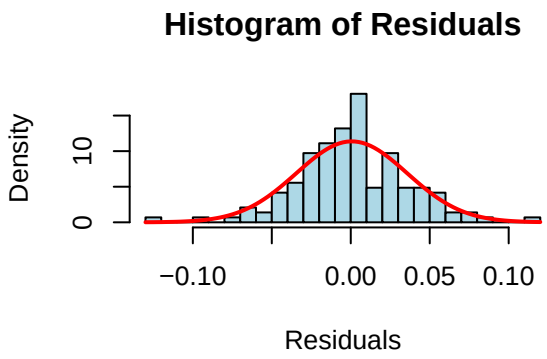
# Residuals over time
plot(final_model$residuals,
     main = "Residuals of Final Model",
     ylab = "Residuals",
     xlab = "Time",
     type = "o",
     col = "darkred",
     pch = 16)
abline(h = 0, col = "blue", lty = 2)
grid()

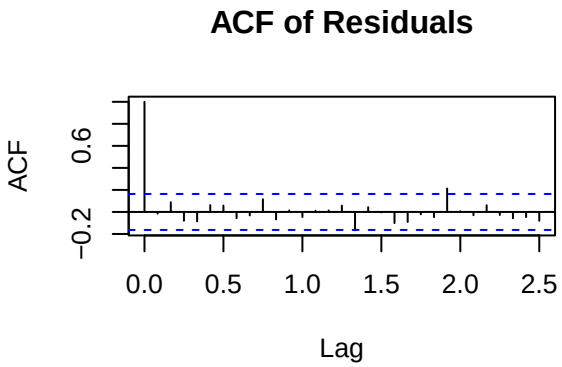
# Histogram of residuals
hist(final_model$residuals,
     main = "Histogram of Residuals",
     xlab = "Residuals",
     col = "lightblue",
     breaks = 20,
     probability = TRUE)
curve(dnorm(x, mean = mean(final_model$residuals),
            sd = sd(final_model$residuals)),
      add = TRUE, col = "red", lwd = 2)

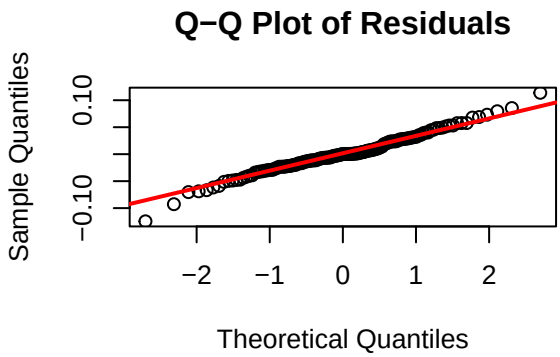
# ACF of residuals
acf(final_model$residuals,
    main = "ACF of Residuals",
    lag.max = 30)

# QQ plot
qqnorm(final_model$residuals, main = "Q-Q Plot of Residuals")
qqline(final_model$residuals, col = "red", lwd = 2)
```










```r
par(mfrow = c(1, 1))

# Shapiro-Wilk test for normality
shapiro_test <- shapiro.test(final_model$residuals)
cat("\nShapiro-Wilk Normality Test:\n")
```

```
##
## Shapiro-Wilk Normality Test:
```

```r
cat("W =", round(shapiro_test$statistic, 4),
    ", p-value =", format.pval(shapiro_test$p.value), "\n")
```

```
## W = 0.9836 , p-value = 0.082379
```

```r
if(shapiro_test$p.value > 0.05) {
  cat("RESULT: Residuals are approximately normally distributed\n")
} else {
  cat("RESULT: Residuals deviate from normality\n")
}
```

```
## RESULT: Residuals are approximately normally distributed
```

```r
# ------------------------------------------------------------
# FORECASTING (Optional)
# ------------------------------------------------------------
cat("\n\nOPTIONAL: FORECASTING\n")
```

```
##
##
## OPTIONAL: FORECASTING
```

```r
cat(rep("-", 50), "\n")
```

```
## - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```

```r
# Forecast next 24 months
forecast_period <- 24
forecast_result <- predict(final_model, n.ahead = forecast_period)

# Convert forecasts back to original scale
forecast_values <- exp(forecast_result$pred)
forecast_se <- exp(forecast_result$se)

# Create time points for forecast
last_year <- end(air_ts)[1]
last_month <- end(air_ts)[2]
forecast_years <- seq(last_year + 1/12, by = 1/12, length.out = forecast_period)

# Plot historical data and forecasts
plot(air_ts,
     xlim = c(1949, 1962),
     ylim = c(0, 700),
     main = "AirPassengers Forecast",
     xlab = "Year",
     ylab = "Passengers",
     col = "darkblue",
     lwd = 2)
grid()

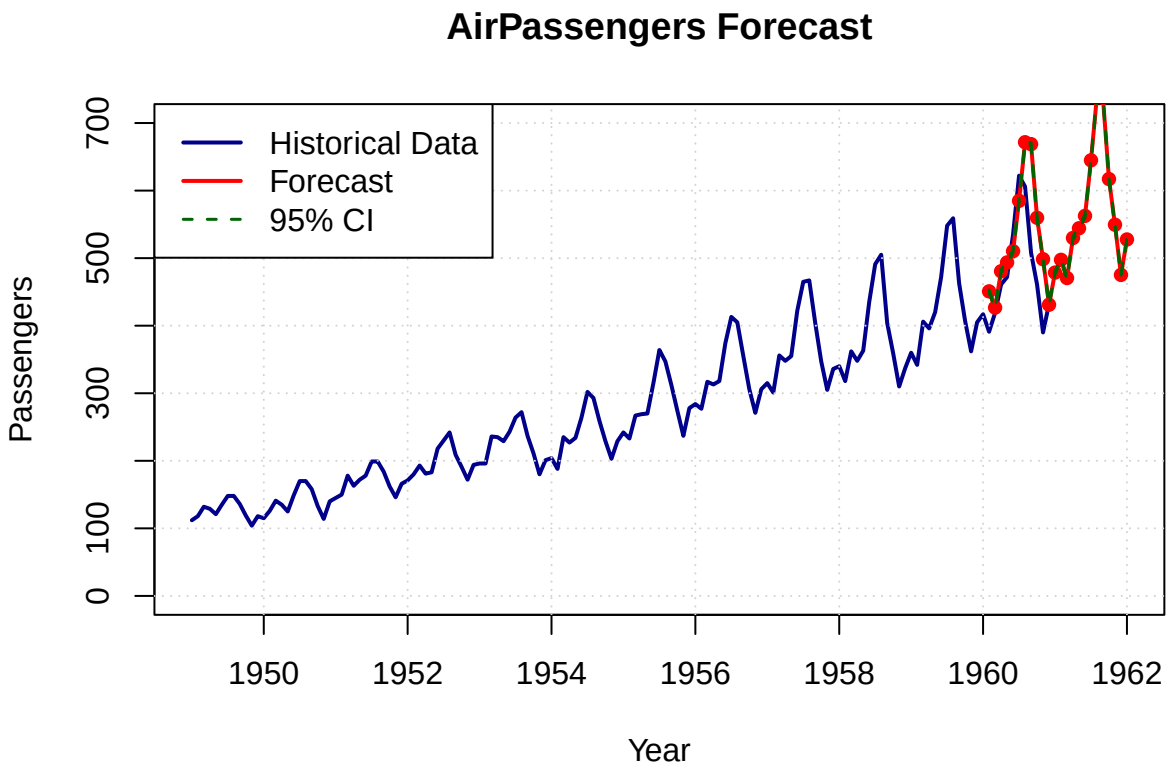
# Add forecasted values
lines(forecast_years, forecast_values, col = "red", lwd = 2)
points(forecast_years, forecast_values, col = "red", pch = 16)

# Add confidence intervals
upper <- forecast_values + 2 * forecast_se
lower <- forecast_values - 2 * forecast_se

lines(forecast_years, upper, col = "darkgreen", lty = 2, lwd = 1.5)
lines(forecast_years, lower, col = "darkgreen", lty = 2, lwd = 1.5)

legend("topleft",
       legend = c("Historical Data", "Forecast", "95% CI"),
       col = c("darkblue", "red", "darkgreen"),
       lty = c(1, 1, 2),
```

```
       lwd = c(2, 2, 1.5),
       bg = "white")
```




```
cat("Forecast for next 24 months created\n")
```

```
## Forecast for next 24 months created
```

```
# ------------------------------------------------------------
# SUMMARY AND CONCLUSIONS
# ------------------------------------------------------------
cat("\n\n", rep("=", 70), "\n", sep="")
```

```
##
##
## ======================================================================
```

```
cat("SUMMARY AND CONCLUSIONS\n")
```

```
## SUMMARY AND CONCLUSIONS
```

```
cat(rep("=", 70), "\n\n")
```

```
## = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = =
```

```r
cat("ANALYSIS RESULTS:\n")
```

```
## ANALYSIS RESULTS:
```

```r
cat(rep("-", 70), "\n")
```

```
## - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```

```r
cat("Analysis Component        | Result\n")
```

```
## Analysis Component        | Result
```

```r
cat(rep("-", 70), "\n")
```

```
## - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```

```r
cat("Data Period               | 1949-1960 (144 months)\n")
```

```
## Data Period               | 1949-1960 (144 months)
```

```r
cat("Original Data Pattern     | Upward trend + strong seasonality\n")
```

```
## Original Data Pattern     | Upward trend + strong seasonality
```

```r
cat("Dominant Components       | TREND and SEASONAL\n")
```

```
## Dominant Components       | TREND and SEASONAL
```

```r
cat("KPSS Test (original)      | p-value =", format.pval(kpss_test$p.value), "\n")
```

```
## KPSS Test (original)      | p-value = 0.01
```

```r
cat("Transformation Applied    | Log + Differencing\n")
```

```
## Transformation Applied    | Log + Differencing
```

```r
cat("Final Model               | ARIMA(1,1,1)(0,1,1)[12]\n")
```

```
## Final Model               | ARIMA(1,1,1)(0,1,1)[12]
```

```r
cat("Model AIC                 |", round(AIC(final_model), 2), "\n")
```

```
## Model AIC                 | -481.9
```

```r
cat("Residual Variance        |", round(final_model$sigma2, 6), "\n")
```

```
## Residual Variance        | 0.001341
```

```r
cat("Ljung-Box Test (lag=20)  | p-value =", format.pval(lb_test$p.value), "\n")
```

```
## Ljung-Box Test (lag=20)  | p-value = 0.6703
```

```r
cat("Shapiro-Wilk Test        | p-value =", format.pval(shapiro_test$p.value), "\n")
```

```
## Shapiro-Wilk Test        | p-value = 0.082379
```

```r
cat(rep("-", 70), "\n\n")
```

```
## - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```

```r
cat("KEY CONCLUSIONS:\n")
```

```
## KEY CONCLUSIONS:
```

```r
cat("1. The AirPassengers data shows strong upward TREND and clear SEASONAL patterns\n")
```

```
## 1. The AirPassengers data shows strong upward TREND and clear SEASONAL patterns
```

```r
cat("2. Log transformation was necessary to stabilize increasing variance\n")
```

```
## 2. Log transformation was necessary to stabilize increasing variance
```

```r
cat("3. The data is non-stationary in its original form (confirmed by KPSS test)\n")
```

```
## 3. The data is non-stationary in its original form (confirmed by KPSS test)
```

```r
cat("4. Seasonal ARIMA model ARIMA(1,1,1)(0,1,1)[12] provides excellent fit\n")
```

```
## 4. Seasonal ARIMA model ARIMA(1,1,1)(0,1,1)[12] provides excellent fit
```

```r
cat("5. Residual diagnostics confirm model adequacy (white noise residuals)\n")
```

```
## 5. Residual diagnostics confirm model adequacy (white noise residuals)
```

```r
cat("6. The model can be used for accurate short-term forecasting\n\n")
```

```
## 6. The model can be used for accurate short-term forecasting
```

```
cat("PRACTICAL IMPLICATIONS:\n")
```

```
## PRACTICAL IMPLICATIONS:
```

```
cat("- The fitted model captures both trend and seasonal patterns in airline data\n")
```

```
## - The fitted model captures both trend and seasonal patterns in airline data
```

```
cat("- Can be used for passenger volume forecasting and capacity planning\n")
```

```
## - Can be used for passenger volume forecasting and capacity planning
```

```
cat("- Helps airlines optimize scheduling and resource allocation\n")
```

```
## - Helps airlines optimize scheduling and resource allocation
```

```
cat("- Provides insights for revenue management and pricing strategies\n\n")
```

```
## - Provides insights for revenue management and pricing strategies
```

```
cat("LIMITATIONS:\n")
```

```
## LIMITATIONS:
```

```
cat("- Model assumes patterns continue (may not capture sudden changes)\n")
```

```
## - Model assumes patterns continue (may not capture sudden changes)
```

```
cat("- External factors (economic conditions, events) not incorporated\n")
```

```
## - External factors (economic conditions, events) not incorporated
```

```
cat("- Forecast uncertainty increases with longer horizons\n")
```

```
## - Forecast uncertainty increases with longer horizons
```

```
cat("- Requires periodic re-estimation with new data\n")
```

```
## - Requires periodic re-estimation with new data
```

```
cat(rep("=", 70), "\n")
```

```
## = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = =
```